AWS LAB REPORT

Title

Hosting a Static Web Application on Amazon EC2 and Implementing Auto Scaling Using Application Load Balancer

1. Aim

To host a static web application on an Amazon EC2 instance using the Nginx web server, create an Amazon Machine Image (AMI), configure a Launch Template, implement an Auto Scaling Group (ASG) with an Application Load Balancer (ALB), and verify automatic scaling using the Stress tool.

2. Objective

- To understand Amazon EC2.
- To host a static web application using Nginx.
- To create an AMI from the configured EC2 instance.
- To create a Launch Template.
- To configure an Auto Scaling Group.
- To configure an Application Load Balancer.
- To distribute traffic across multiple EC2 instances.
- To test automatic scaling by generating CPU load using the Stress tool.

3. Introduction

Amazon Elastic Compute Cloud (Amazon EC2) is a web service provided by Amazon Web Services (AWS) that allows users to create virtual servers in the cloud. These virtual servers are called EC2 instances. EC2 provides scalable computing capacity, enabling users to deploy applications without purchasing physical hardware.

In this experiment, a static web application is hosted on an Ubuntu EC2 instance using the Nginx web server. After successful deployment, an Amazon Machine Image (AMI) is created to capture the configured server. The AMI is then used to create a Launch Template, which acts as a blueprint for launching identical EC2 instances. An Auto Scaling Group (ASG) is configured to automatically add or remove EC2 instances based on CPU utilization. An Application Load Balancer (ALB) distributes incoming user requests among healthy EC2 instances, ensuring high availability and improved performance. Finally, the Stress tool is used to generate CPU load and verify the Auto Scaling functionality.

4. Services Used

AWS Service	Purpose
Amazon EC2	To host the web application
Amazon AMI	To create an image of the configured instance
Launch Template	To define instance configuration
Auto Scaling Group (ASG)	To automatically scale EC2 instances
Application Load Balancer (ALB)	To distribute traffic among instances
Target Group	To register and monitor EC2 instances

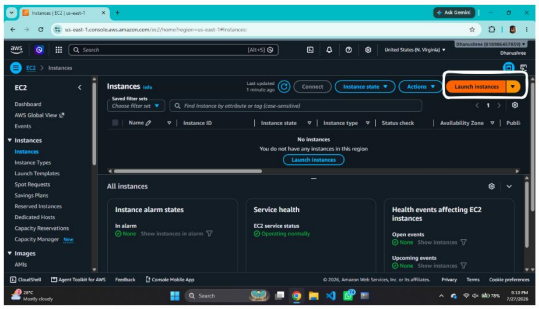
7. Execution Steps

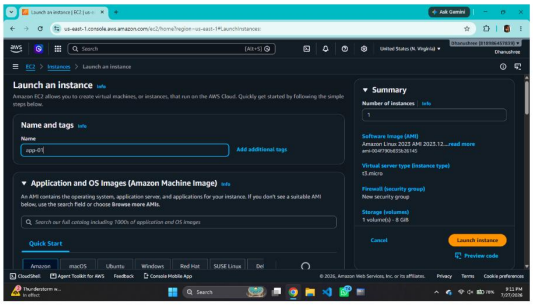
Step 1: Launch an EC2 Instance

An EC2 instance is a virtual machine in AWS. We need an EC2 instance to host our website.

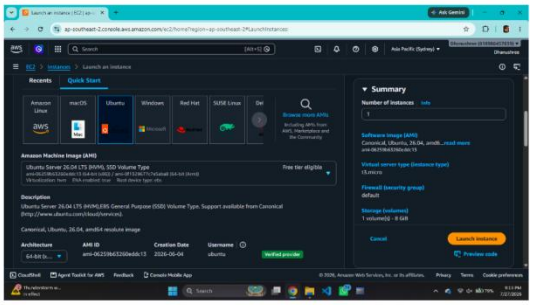
Procedure

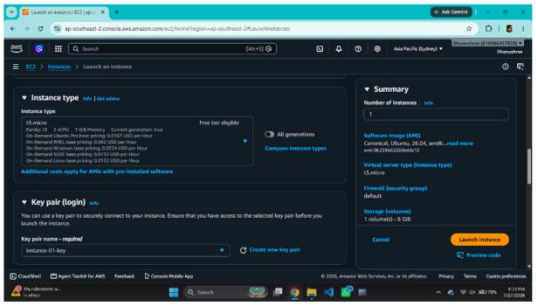
1. Go to EC2 service in AWS
2. Click on instances in EC2 dashboard
3. Click on launch instance
4. Provide the name for instance

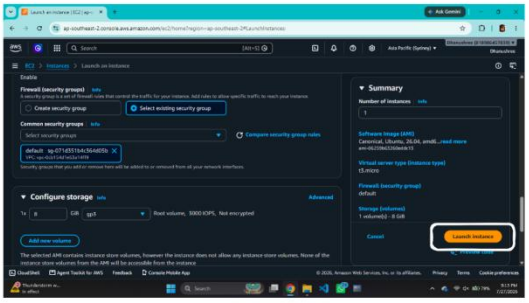




5. In OS select "ubuntu" as an OS
6. In instance type select "t2.micro" as instance type
7. In key pair
   -> Click on existing key pair
8. Under networking, in security groups click on select existing security group and select as default
9. In storage configure the storage according to the requirement under "gp2" or "gp3" (general purpose)
10. Click on launch instance







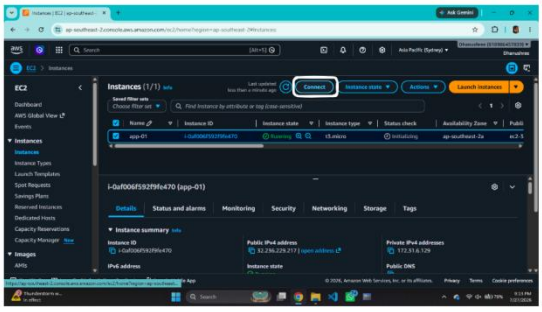
Step 2: Connect to the EC2 Instance

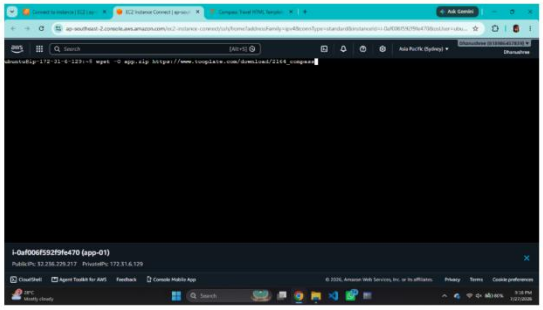
We need to log in to the server to install software and deploy the application.

Procedure

1. Once after launching the instance, go to instance dashboard and under instances make sure the instance is in "running state"
2. Select the instance and click on connect
3. Select "EC2 instance connect" as connection method and click on connect
4. Make sure EC2 instance is successfully connected
5.Go to tooplate website and get into any template and copy the download link URL
6.In instance download the template via "wget command"

Command: wget -O <zip-filename>.zip <download-link-URL>



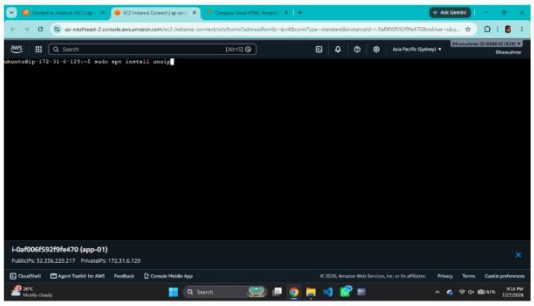


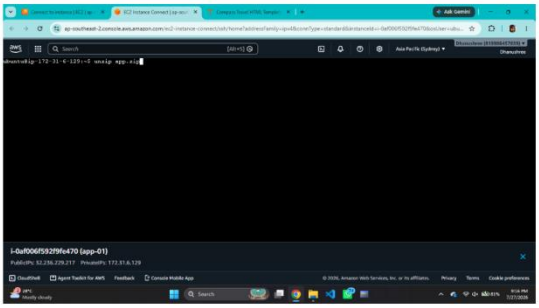
`

7.Install unzip and then unzip the zip file

Command: sudo apt install unzip

unzip <zip-filename>.zip



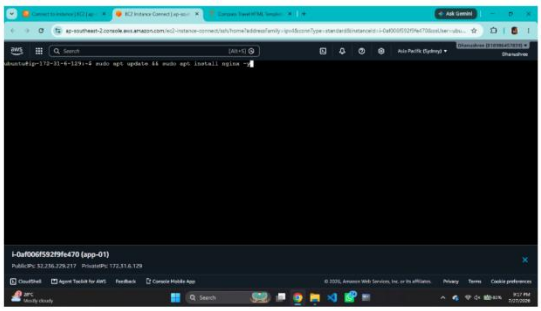


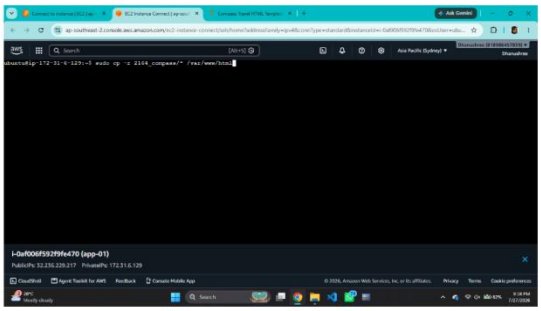
8. Install nginx web server

    Command: sudo apt update && sudo apt install nginx -y

9. Verify nginx is installed or not by giving the command -"nginx -v"
10. Copy files from extracted zip folder to nginx path (`/var/www/html`)

    Command: sudo cp -r <source-folder-path>/* /var/www/html

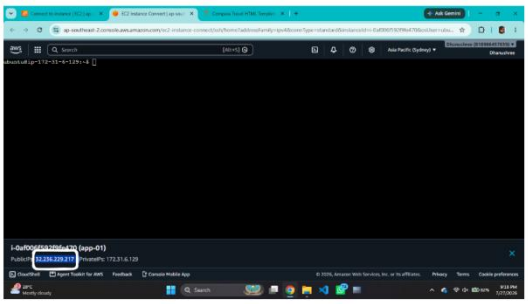


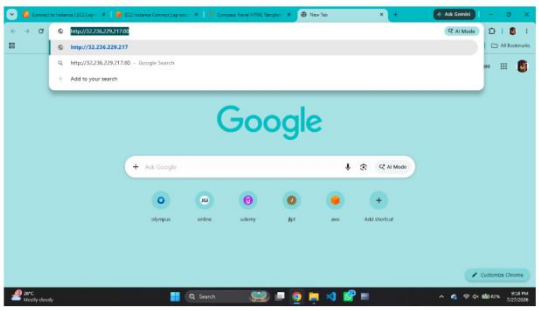


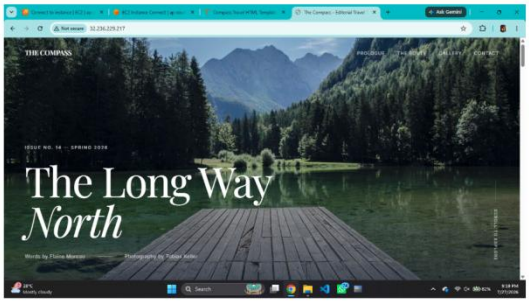
11. In security group make sure 80 port number is enabled as it is default port for nginx and http
12. Copy the public IP of instance and paste it in browser with port 80

    Sample URL: http://<public-IP-instance>:80`

13. Make sure application is accessible over the network







Step 8: Create an Amazon Machine Image (AMI)

Instead of configuring every EC2 instance manually, AWS allows us to create an AMI (Amazon Machine Image).

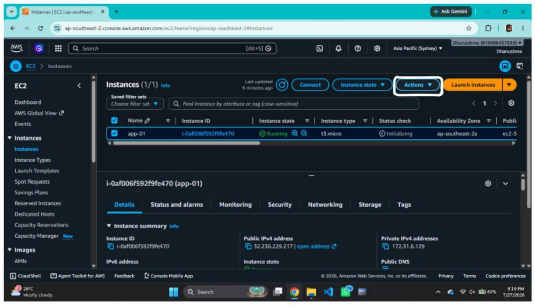
The AMI contains:

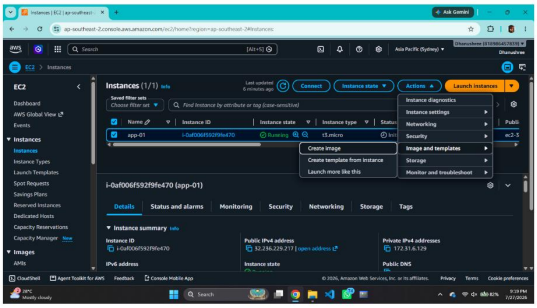
- Operating System
- Installed Nginx
- Hosted Website
- Configurations

Future EC2 instances launched from this AMI will already have the application installed.

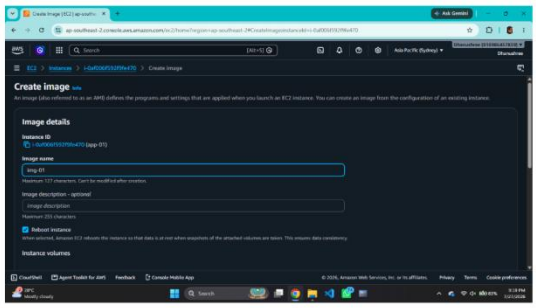
Procedure

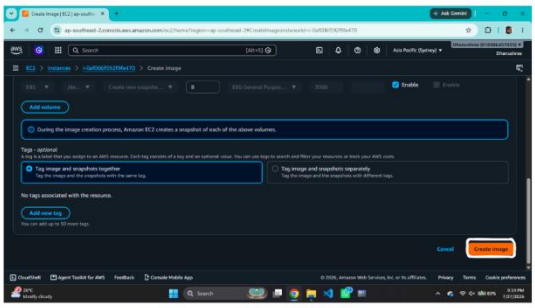
1. Go to instance dashboard and select the instance and click on actions

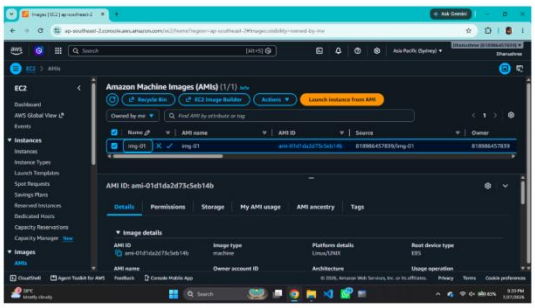




2. Under image and templates click on create image and provide the image name and click on create image
3. Under images, click on AMI in instance dashboard and make sure AMI is in available state







Step 9: Create a Launch Template

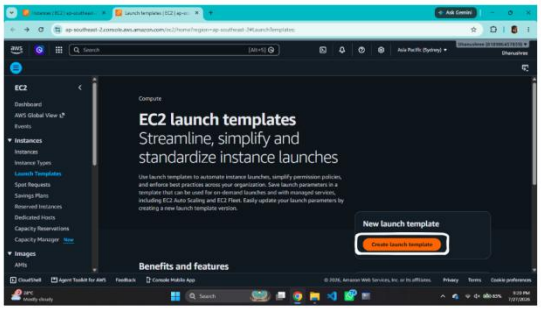
A Launch Template acts as a blueprint for launching EC2 instances.
It stores:

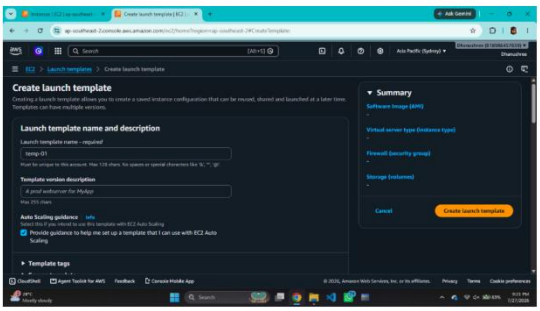
- AMI
- Instance Type
- Security Group
- Storage
- Key Pair

Whenever Auto Scaling creates a new instance, it uses this template.

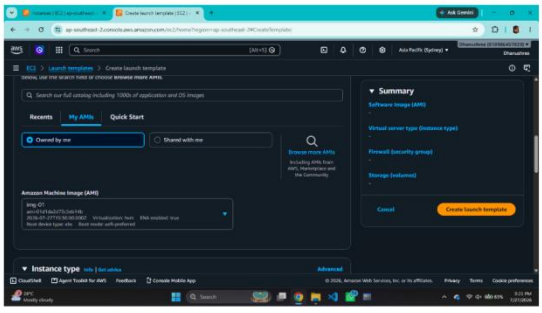
Procedure

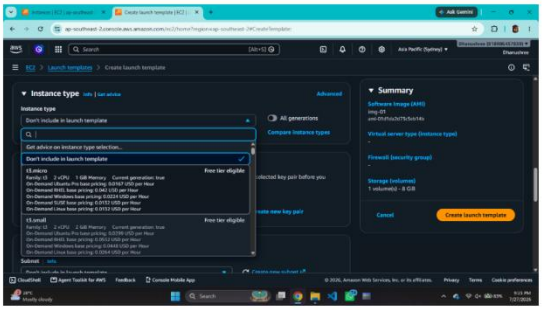
1. In instance dashboard, click on launch templates
2. Click on create launch templates and provide launch template name
3. Tick the auto scaling guidance option to enable launch template for ASG



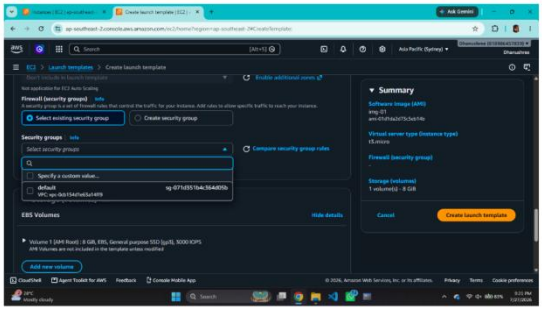


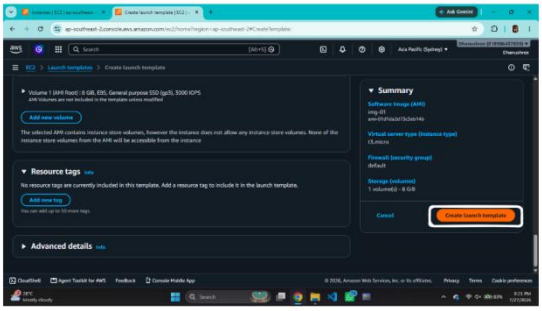
4. Select the created AMI
5. Select instance size as "t3.micro"





6. Select default security group and key pair
7. Click on launch template



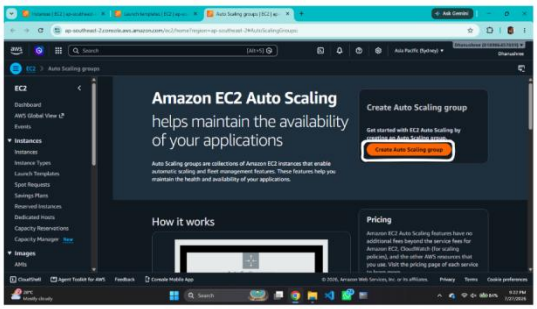


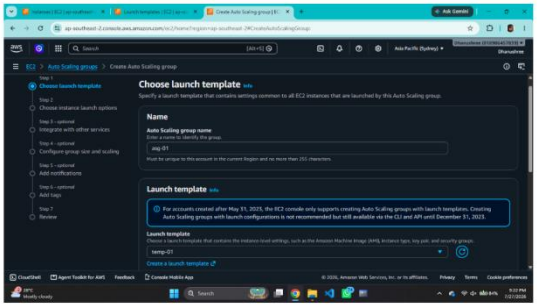
Step 10: Create an Auto Scaling Group

An Auto Scaling Group automatically launches or terminates EC2 instances depending on traffic or CPU utilization.

Procedure

1. In instance dashboard, click on auto scaling group and click on create auto scaling group
2. Provide name for auto scaling group and select the created launch template and click on next



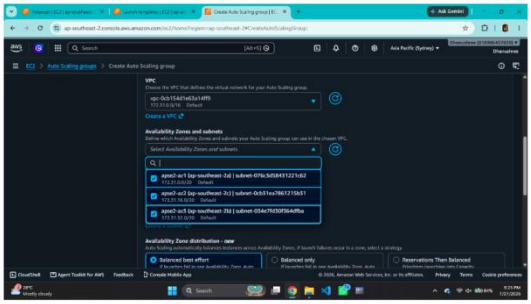


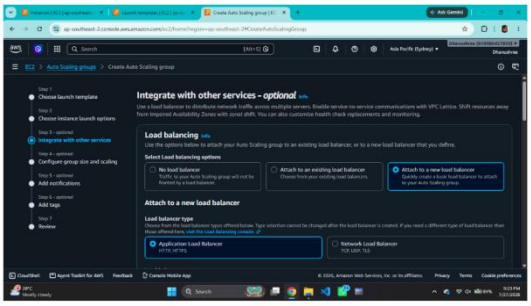
2. In availability zones select 3 availability zone and in availability zone distribution select as "balanced best effort" and click on next

Create a load balancer

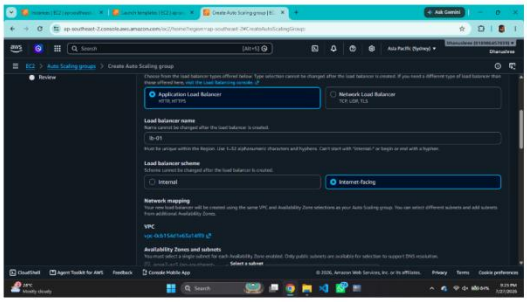
If multiple EC2 instances exist, users should not connect directly to them.
Instead, users access the Load Balancer, which distributes incoming requests evenly among all healthy instances.

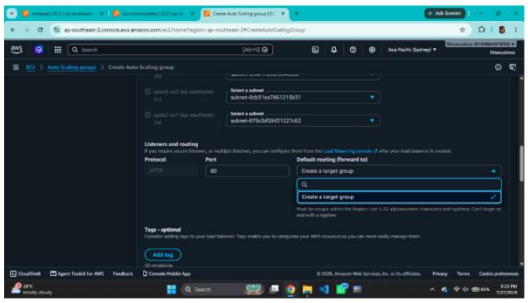
1. Click on attach to a new load balancer
2. Select load balancer type as "application load balancer"



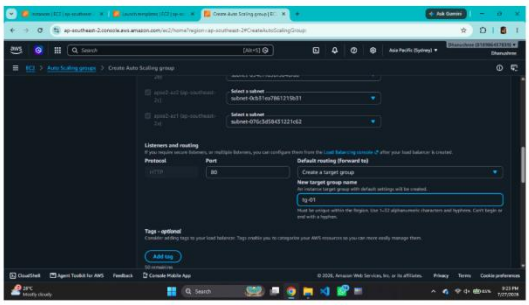


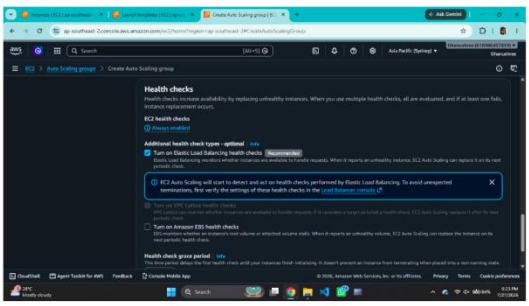
3. Provide the name for load balancer
4. Select scheme as "internet facing"
5. Under listeners and routing, click on create target group and provide name for target group





6. Enable health checks and click on next

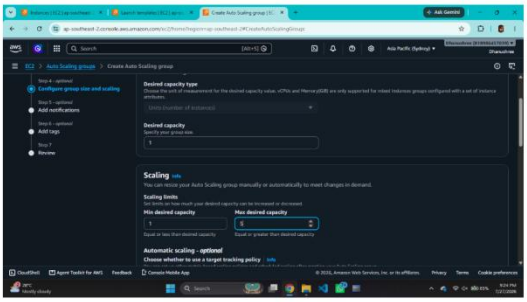


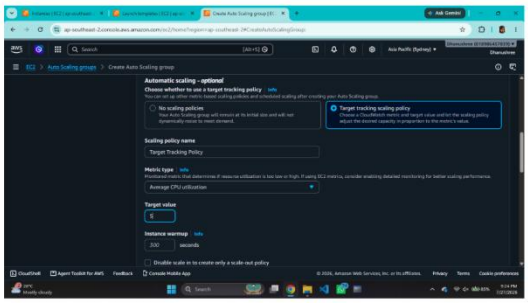


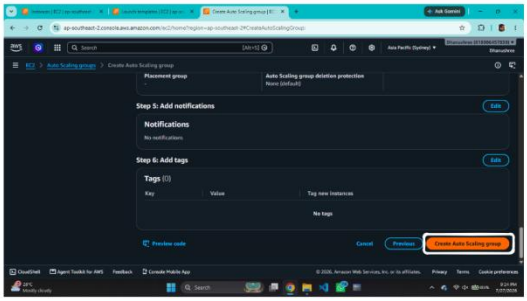
Configure size of ASG

1. Select desired instance as 1 and select min and max instances (Ex: min : 1 , max: 5)
2. Under target tracking scaling policy, click on create target tracking scaling policy and select CPU utilization as metric and target value as "5"
3. Click on next

Skip the notification and tags step and click on create auto scaling groups







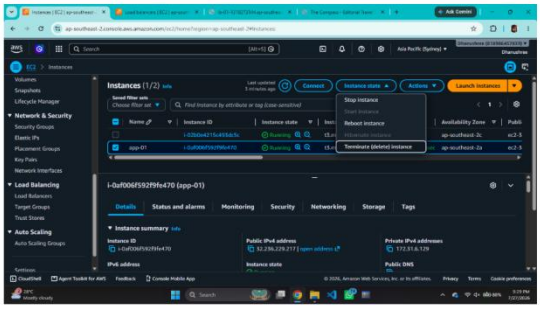
Step 14: Install the Stress Tool

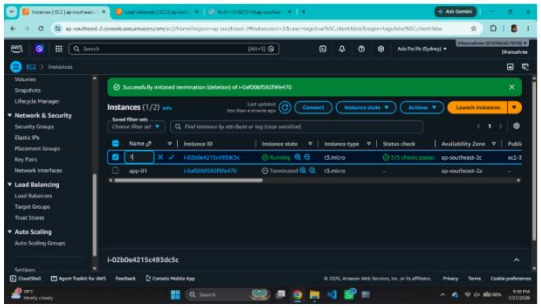
To test Auto Scaling, we need to artificially increase CPU usage.
Connect to the running EC2 instance.

Procedure

1.Delete the old instance

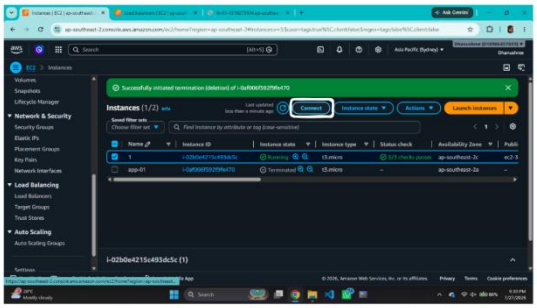
2.Rename the new instance created by AMI

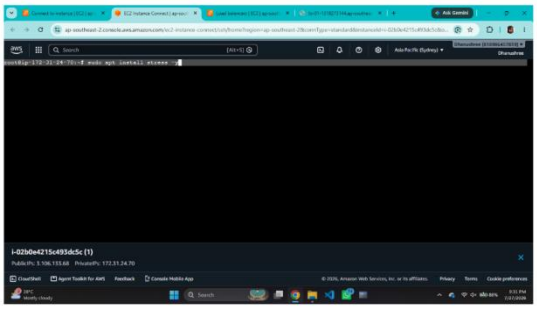




3.Connect to the desired instance and install stress command

Command: sudo apt install stress -y





4. Provide stress for EC2 instance

Command: stress --cpu 2 --timeout 300

Explanation

--cpu 2 creates two CPU workers to consume CPU resources.

--timeout 300 runs the test for 300 seconds (5 minutes).

This increases CPU utilization above the 5% threshold configured in the Auto Scaling policy.

5. Refresh the instance state over the period of the time instance will be scaled up

